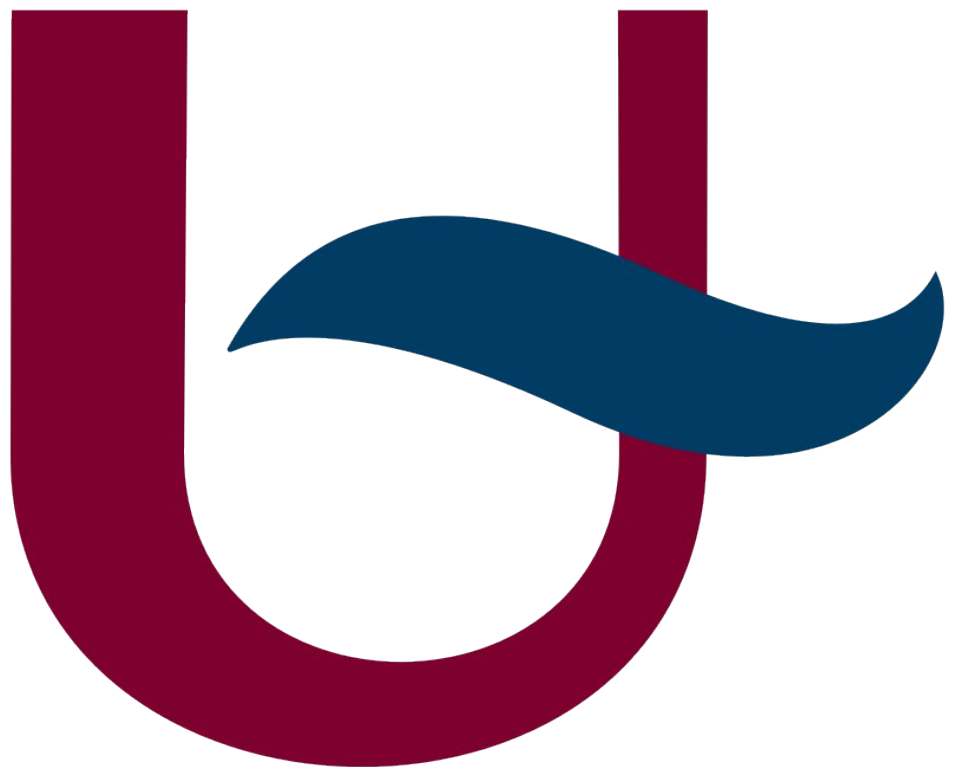


I-MAP: LOGBOOK (AI USAGE)

TSMK



LLM request

△ [Claude](#) was used for this prompt.

Is it possible to get something like this when flashing my platform.IO project?

RAM: [=] 8.3% (used 27196 bytes from 327680 bytes)

Flash: [=] 16.2% (used 267988 bytes from 1655808 bytes)

LLM resopnse

Yes! Add this to your `platformio.ini`:

ini

```
[env:teensy41]
```

```
platform = teensy
```

```
board = teensy41
```

```
framework = arduino
```

```
check_tool = clangtidy
```

```
extra_scripts = pre:print_size.py
```

Actually, the simpler fix is just a `custom_size_format` option — but unfortunately Teensy's platform doesn't support that directly.

The most reliable way is a small **extra script**. Create a file called `print_size.py` in your project root:

python

```
import("env")
```

```
import subprocess
```

```
import os
```

```
def print_size(source, target, env):
```

```
    elf = str(target[0])
```

```
    size_tool = os.path.join(
```

```
        env.subst("$PROJECT_PACKAGES_DIR"),
```

```
        "toolchain-gccarmnoneabi-teensy", "bin", "arm-none-eabi-size"
```

```
    )
```

```
    result = subprocess.run([size_tool, elf], capture_output=True, text=True)
```

```
    lines = result.stdout.strip().split("\n")
```

```
    if len(lines) >= 2:
```

```
        parts = lines[1].split()
```

```
        text, data, bss = int(parts[0]), int(parts[1]), int(parts[2])
```

```
        flash_used = text + data
```

```
        ram_used = data + bss
```

```
        flash_total = 7 * 1024 * 1024 + 750 * 1024 # 7.75MB
```

```
        ram_total = 512 * 1024
```

```
    def bar(used, total, width=10):
```

```
        filled = int(round(used / total * width))
```

```
        return "[" + "=" * filled + " " * (width - filled) + "]"
```

```
    print(f"RAM: {bar(ram_used, ram_total)} {ram_used/ram_total*100:.1f}%  
(used {ram_used} bytes from {ram_total} bytes)")
```

```
    print(f"Flash: {bar(flash_used, flash_total)} {flash_used/flash_total*100:.1f}%  
(used {flash_used} bytes from {flash_total} bytes)")
```

```
env.AddPostAction("$BUILD_DIR/${PROGNAME}.elf", print_size)
```

Then in your `platformio.ini`, just add the extra script line:

ini

```
[env:teensy41]
```

```
platform = teensy
```

```
board = teensy41
```

```
framework = arduino
```

```
extra_scripts = post:print_size.py
```

Now every `pio run` or `pio run -t upload` will print:

```
RAM: [=      ] 8.3% (used 43520 bytes from 524288 bytes)
```

```
Flash: [=     ] 16.2% (used 267988 bytes from 8114176 bytes)
```

The script hooks into the post-build step so it runs automatically after linking, before uploading.